Bacterial phylogenomics: hands-on handout

Nov 8, 2026 · @Haim

Two tracks run through every step: **A** your own genomes, **B** the 63-genome *P. syringae* example set (outgroup *P. fluorescens* GCF_900215245.1). Lines starting with `#` are comments; replace `list.txt`-style file names with your own.

0. Setup

Track B: get the workshop repository, which holds the environments, scripts and example files used below (only the large Microbializer results are separate, see below). Track A: only your genome FASTA files are needed, but take the environments and scripts from the repository too.

```
# in any folder you like (needs git; no git: GitHub Code button > Download
ZIP, then unzip)
git clone https://github.com/HaimAshk/bacterial-phylogenomics-workshop.git
cd bacterial-phylogenomics-workshop     # the folder git just created; paths
below are relative to it
ls                                      # README.md  data/  envs/  scripts/
slides/
```

No conda yet? Install Miniforge (free, uses the conda-forge channel): https://github.com/conda-forge/miniforge . After installing, open a new terminal. If `conda` is still not found, run `conda init` from the install folder (for example `~/miniforge3/bin/conda init`) and open a new terminal again. On Windows, use WSL (Ubuntu) and install Miniforge inside it.

```
# run from inside bacterial-phylogenomics-workshop/; -p puts each environment
in a folder next to the repository files
# environment 1: genome comparison (steps 1-4)
conda env create -f envs/phylo-genomes.yml -p ./conda_envs/phylo-genomes

# environment 2: gain/loss, co-evolution and GWAS (steps 5-6)
conda env create -f envs/phylo-gwas.yml -p ./conda_envs/phylo-gwas
conda activate ./conda_envs/phylo-gwas
Rscript -e
'remotes::install_git("https://github.com/caitiecollins/treeWAS.git",
build_vignettes=FALSE)'
gloome --help       # check that command-line GLOOME runs (the web server
needs no install)
```

Two environments, because GLOOME needs `numpy<2`, which can conflict with the genome tools. Activate by path. Steps 1-4: `conda activate ./conda_envs/phylo-genomes`; steps 5-6: `conda activate ./conda_envs/phylo-gwas` (from the repository folder; when you are inside `work/`, use `../conda_envs/...`). The two `.yml` files are in the `envs/` folder of the repository.

Next, make one working folder and copy the files into it. The commands below use bare file names and are written to run inside it:

```
# run from inside bacterial-phylogenomics-workshop/ (after the conda lines
above)
mkdir -p work && cd work
cp ../data/pseudomonas/* ../data/toy_example/* ../scripts/* .
ls                  # accessions_63.txt  toy2_tree.nwk  phylo_plots.py
run_treewas.R ...
```

Keep your results in `work/` too. The repository folders (`data/`, `scripts/`) stay untouched, so you can always copy the originals again.

Example-set files (track B), now in `work/`: `accessions_63.txt`, `pseudomonas_tree_for_gloome.nwk`, `pseudomonas_phyletic_pattern_ALL.fas` (26,703 families), `pseudomonas_phyletic_pattern.fas` (11,805 variable families), `pseudomonas_phyletic_positions.tsv`, the Microbializer outputs `orthogroups_annotated.csv`, `ani_pairwise_values.csv` and `final_species_tree.newick` (used by the plots in step 4), and `toy2_*` (small simulated treeWAS example). The full Microbializer results (about 400 MB, a `.tar.gz`) are optional, on Keeper; extract them with `tar -xzf`.

1. Get genomes and metadata

Track A: put your genomes in `fna/` (one FASTA per genome, file name = genome name) and skip to the last two lines. Track B: download from NCBI.

```
datasets download genome accession --inputfile accessions_63.txt \
    --include genome --filename genomes.zip
unzip -o genomes.zip
mkdir -p fna
find ncbi_dataset -name '*_genomic.fna' -exec cp {} fna/ \;

ls $PWD/fna/*.fna > list.txt                    # one path per line (skani,
dRep)
wc -l list.txt                                  # expect 63

# metadata: host, isolation source, country, date, assembly level (writes
metadata.tsv)
bash ncbi_metadata.sh accessions_63.txt
head -3 metadata.tsv
```

Look at `metadata.tsv` before going on: the phenotype for the GWAS step (for example host plant or isolation source) comes from here.

2. Genome similarity (ANI and k-mer distances)

skani gives pairwise ANI in seconds. Pairs below roughly 80% ANI are not reported, so an outgroup may be missing from the table.

```
skani triangle -l list.txt -E -t 8 -o skani_edges.tsv     # one row per
genome pair
skani triangle -l list.txt -t 8 -o skani_matrix.txt       # lower-triangular
matrix
head skani_edges.tsv
```

PopPUNK separates core distance (sequence divergence) from accessory distance (gene content). It was designed for within-species data, so on a multi-phylogroup set read the two distances rather than the cluster assignments.

```
# two tab-separated columns: genome name, path
paste <(xargs -n1 basename -s .fna < list.txt) list.txt > pp_input.txt

poppunk --create-db --r-files pp_input.txt --output pp_db --threads 8
poppunk --fit-model bgmm --ref-db pp_db --output pp_fit --K 4
poppunk_visualise --ref-db pp_db --model-dir pp_fit --output pp_vis --
microreact
```

dRep clusters genomes into species- or strain-level groups (primary Mash 90%, secondary ANI 95% by default; use 98–99% for strains).

```
dRep dereplicate drep_out -g fna/*.fna -pa 0.9 -sa 0.95 -p 8 --
ignoreGenomeQuality
```

Question to answer: which pairs have high ANI but a large accessory distance?

3. Pangenome: orthogroups and the presence/absence matrix

Microbializer (web server; address on the workshop page).

```
# track B: builds the input zip from the accession list (outgroup included)
bash make_microbializer_zip.sh accessions_63.txt microbializer_input
# track A: simply zip your own FASTA files
zip microbializer_input.zip fna/*.fna
```

1. Upload the zip, give an e-mail address, submit. Run time grows with the number of genomes, so start it first.
2. From the results download `orthogroups_annotated.csv` (matrix of gene IDs, one row per family), `ani_pairwise_values.csv` and `final_species_tree.newick`.
3. Track B: these three files are already in work/ (step 0), so you can skip the run.

PanGene-O-Meter (gene-content similarity, GCS) needs the presence/absence of orthology groups per genome. Here we give it the Microbializer phyletic pattern, so no proteins are extracted and no genes are clustered. It has its own environment (Perl, DIAMOND, pyrodigal), separate from the two above. Install it once, outside the workshop repository (for example next to it):

```
git clone https://github.com/HaimAshk/PanGene-O-Meter.git
cd PanGene-O-Meter
conda env create -p ./conda_env/pangeneometer -f environment.yml    # or
without -p: a named environment, pangeneometer
conda activate ./conda_env/pangeneometer
perl PanGeneOmeter/PanGeneOmeter.pl --help
diamond version
```

Run it on the phyletic pattern (from `work/`, using the full path to the script; the `PGOM_*.pm` files must stay next to `PanGeneOmeter.pl`):

```
perl /full/path/to/PanGene-O-Meter/PanGeneOmeter/PanGeneOmeter.pl \
     --phyletic_pattern_file pseudomonas_phyletic_pattern_ALL.fas \
     --outDir pgom_pseudomonas --threads 4
```

The output directory has the pairwise gene-content distances, the clusters and the representatives. Genomes are renamed to S1 … SN during the run and renamed back in the result tables (`genome_names.map.txt` has the mapping). The matrix format is detected automatically (Microbializer `phyletic_patterns.fas`, Panaroo or Roary Rtab, or a `*.01.csv`); it needs at least 2 genomes and only 0/1 values. Other inputs: GenBank files (`--in_gbk_list_file`) or genome FASTA files (`--in_genome_list_file list --input_type fna`, genes predicted with pyrodigal). Use the DIAMOND version that comes with the environment: v2.1.24 and v2.1.25 do not work at the moment.

Check the numbers against the example: 63 genomes, 26,703 families, 2,362 core (all 63), 12,536 singletons, 11,805 variable (2–62 genomes).

4. Plots from the Microbializer output

One script, `phylo_plots.py`, makes four figures (tested on the example set):

```
python phylo_plots.py --og orthogroups_annotated.csv \
    --ani ani_pairwise_values.csv \
    --tree final_species_tree.newick --out plots/
```

File	Shows	What to look for (example set)
`gcs_vs_ani.png`	gene-content similarity (Jaccard and overlap) against ANI, per genome pair	Pearson r = 0.79 (Jaccard), 0.75 (overlap); at similar ANI the spread in gene content is large, especially above 95%
`saturation.png`	pangenome and core size as genomes are added (100 random orders)	the pangenome keeps growing; Heaps' exponent γ ≈ 0.43, so the pangenome is open
`ushape.png`	number of families against the number of genomes carrying them (log y)	many rare families, a second peak at 63 (core)
`tree_matrix.png`	species tree next to the presence/absence matrix of the 11,805 variable families	blocks of families that follow clades, and scattered patterns that do not (homoplasy)

Track A needs the same three files from your own Microbializer run. Without ANI or a tree, leave out those flags and the matching plots are skipped.

Extensions to try: colour the GCS vs ANI points by whether the pair is within a phylogroup; plot the GCS of one genome to all others against its ANI to the same genomes; add `metadata.tsv` columns as coloured bars next to the tree.

5. Gain and loss, co-evolution: GLOOME and CoPAP

Input is the phyletic pattern (FASTA of 0/1 per genome, one position per family) and a Newick tree. Use the unfiltered pattern: GLOOME estimates rates from all positions, and CoPAP tests only the variable ones.

```
# toy example (seconds; includes the CoPAP co-evolution table)
gloome --msa_file toy2_phyletic_pattern.fas --tree_file toy2_tree.nwk --
out_dir gloome_toy
ls gloome_toy/*/                # Branches.tsv, PosteriorRates.tsv,
Coevolution.tsv, ...

# Pseudomonas, rooted on the P. fluorescens outgroup, CoPAP switched off (too
heavy for the full set)
gloome --msa_file pseudomonas_phyletic_pattern_ALL.fas \
       --tree_file pseudomonas_tree_for_gloome.nwk \
       --rooting_method outgroup --leaf GCF_900215245.1 \
       --is_do_not_use_copap 1 \
       --out_dir gloome_pseudomonas
ls gloome_pseudomonas/*/        # Branches.tsv, Nodes.tsv,
BranchPositionProbabilities.tsv,
                                # ParsimonyAndHomoplasyScores.tsv,
TreeAttributes.tsv, ...
```

CoPAP is computationally heavy, so in the workshop it runs on the toy example only. The GLOOME web server takes the same two files for small inputs (it does not accept the full Pseudomonas data). Checks before running:

- Newick starts with `(`, ends with `;`, balanced parentheses; strip support values if the tree is rejected.
- The taxon names in the FASTA headers and in the tree are identical.
- Default rooting is midpoint; choose the outgroup rooting when you have an outgroup.

Look at: TreeAttributes.tsv (π1, the stationary frequency of presence, about 0.08 on the Pseudomonas set so genes are mostly absent; the Γ α value, about 0.40, so rates differ strongly between families; and the four rate categories); the expected number of gains per family (families with more than one gain are homoplasic); and, for the toy example, the CoPAP table of co-evolving family pairs (Coevolution.tsv). Map the OG numbers to functions with `pseudomonas_phyletic_positions.tsv` and the annotation column of `orthogroups_annotated.csv`.

6. Gene-content GWAS: treeWAS and pyseer

You need a binary or continuous phenotype per genome (from `metadata.tsv`, for example host of isolation) and the presence/absence matrix. Toy files: `toy2_PA_matrix.tsv` (genomes in rows, families in columns), `toy2_trait.tsv`, `toy2_tree.nwk`.

treeWAS (R) simulates a null along the tree and scores terminal, simultaneous and subsequent associations:

```
Rscript run_treewas.R          # runs the toy example; read the script, it is
short
```

For your own data, edit the three file names at the top of `run_treewas.R`; the matrix must be genomes × families with row names equal to the tree tips.

pyseer corrects for population structure with a linear mixed model. It expects families in rows and genomes in columns (Rtab), so the matrix is transposed first. The two helper scripts `phylogeny_distance.py` and `count_patterns.py` are not installed by pip, so download them once into `work/`:

```
# once: the two helper scripts (pyseer GitHub) and the tree library they need
curl -LO
https://raw.githubusercontent.com/mgalardini/pyseer/master/scripts/phylogeny_
distance.py
curl -LO
https://raw.githubusercontent.com/mgalardini/pyseer/master/scripts/count_patt
erns.py
python -c "import dendropy" || pip install dendropy

# input files: genomes x families -> families x genomes; phenotype file with
a header
python -c "import pandas as pd; pd.read_csv('toy2_PA_matrix.tsv', sep='\t',
index_col=0).T.rename_axis('Gene').to_csv('toy2.Rtab', sep='\t')"
cut -f1,2 toy2_trait.tsv | sed '1s/.*/samples\tphenotype/' >
toy2_trait_pyseer.tsv

# tree distances, the mixed-model GWAS, number of unique patterns
python phylogeny_distance.py --lmm toy2_tree.nwk > phylo_dist.tsv
pyseer --lmm --phenotypes toy2_trait_pyseer.tsv --pres toy2.Rtab \
       --similarity phylo_dist.tsv --output-patterns patterns.txt > gwas.txt
python count_patterns.py patterns.txt     # prints Patterns and the
Bonferroni Threshold
head gwas.txt
```

`toy2_trait_pyseer.tsv` has two columns, `samples` and the phenotype, with a header line; the `cut` and `sed` line above builds it from `toy2_trait.tsv`. In `gwas.txt` read the `lrt-pvalue` column and compare it with the threshold printed by `count_patterns.py` (the notes column may say `bad-chisq` for rare families; that is common). Compare the hits with `toy2_truth.tsv`, which lists the families that really carry the signal, and ask: which method finds them, and which ones are false positives from population structure?

Things to explore

Pick any of these, in any order. Work in `work/` and bring questions or error messages to the group.

- **Homoplasy:** which families are scattered across the tree, and which follow the clades? Use `ParsimonyAndHomoplasyScores.tsv` from the GLOOME Pseudomonas run and `tree_matrix.png`.
- **Gains and losses:** on which branches do gains and losses cluster? Start with `Branches.tsv` and `BranchPositionProbabilities.tsv`, and the rate parameters in `TreeAttributes.tsv`.
- **Toy example:** which method finds the planted families, and which hits are decoys? Compare the treeWAS hits with `toy2_truth.tsv`.
- **Genome pairs:** which pairs have high ANI but very different gene content? Use `gcs_vs_ani.png`.
- **Your own data:** put your genomes in `fna/` and follow steps 1 to 5.

If a command fails, run it with `--help` first; tool options change between versions. Bring the error message, not a description of it.